Question

Analyze the architecture and design of the Java classes in the new notification system, then answer the following and the output will be

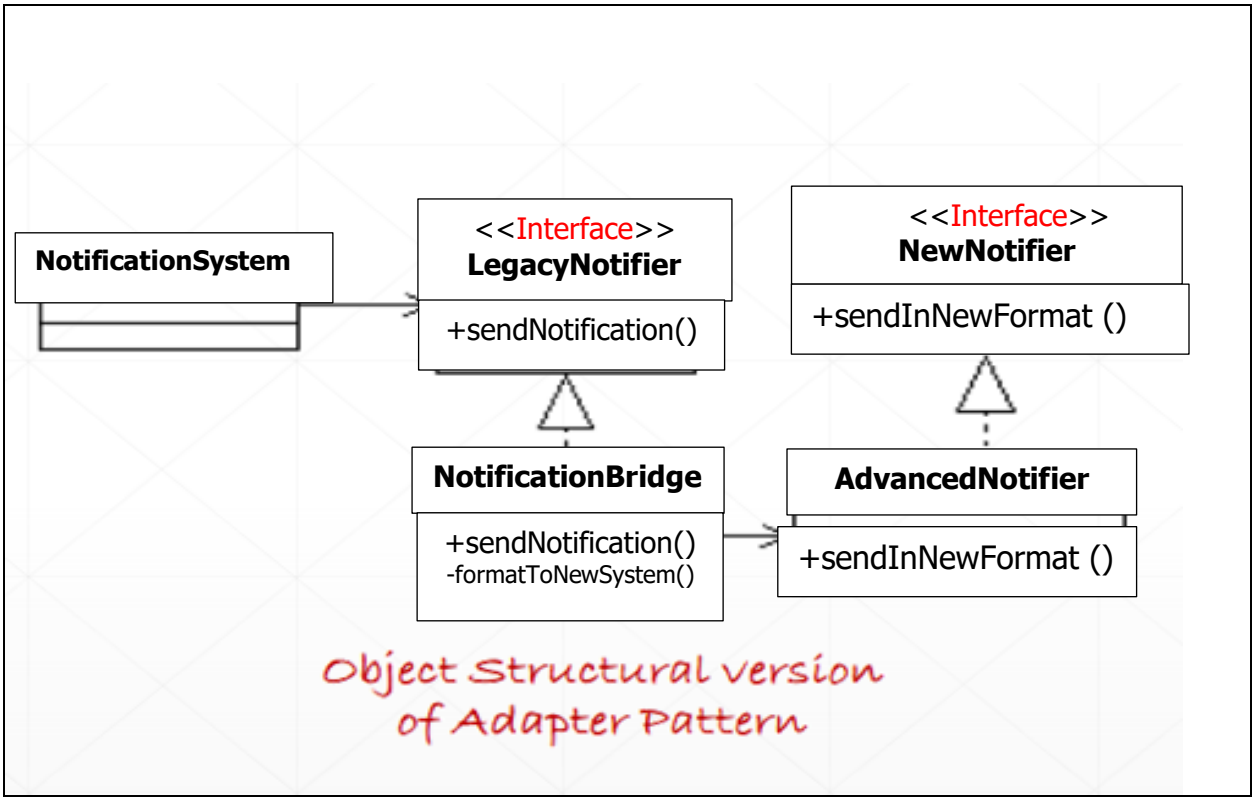
Notifying with message: {"notification": "System is operational"}

A. Identify the design pattern used in this example. If its Adapter design pattern the code below is Class-based Structural Version **OR** Object-based Structural Version?

B. Write the code of the other Object-based Structural Version if the original code if Class-based Structural Version? And vice versa?

C. Draw a UML diagram for the given Java class design.

D. Mention one advantage and one disadvantage of the identified design pattern.



Object Adapter:

```
interface LegacyNotifier {
    void sendNotification(String message);
}

class NotificationBridge implements LegacyNotifier {
    private final NewNotifier newNotifier;

    NotificationBridge(NewNotifier newNotifier) {
        this.newNotifier = newNotifier;
    }

    @Override
    public void sendNotification(String message) {
        String formattedMessage = formatToNewSystem(message);
        newNotifier.notifyInNewSystem(formattedMessage);
    }

    private String formatToNewSystem(String message) {
        return "{\"notification\": \"" + message + "\"}";
    }
}

interface NewNotifier {
    void notifyInNewSystem(String formattedMessage);
}
```

```
class AdvancedNotifier implements NewNotifier {
    @Override
    public void notifyInNewSystem(String formattedMessage) {
      System.out.println("Notifying with message: " + formattedMessage);
    }
}

public class NotificationSystem {
 public static void main(String[] args) {
    NewNotifier advancedNotifier = new AdvancedNotifier();
    LegacyNotifier notifier = new NotificationBridg(advancedNotifier);
        notifier.sendNotification("System is operational");
    }
}
```

Classes Adapter:

- **NotifierBridge** now:
  - Extends **AdvancedNotifier**, meaning it inherits the `sendInNewFormat` method from **AdvancedNotifier**.
  - Implements the **OldNotifier** interface, meaning it provides an implementation for the `notify` method.
- **notify** method in **NotifierBridge**:
  - Formats the message to JSON format.
  - Calls `sendInNewFormat` (inherited from **AdvancedNotifier**) to send the notification in the new format.

```
// Old notifier interface for the existing notification mechanism
interface OldNotifier {
    void notify(String message);
}

// New notifier interface for the advanced notification mechanism
 interface NewNotifier {
    void sendInNewFormat(String formattedMessage);
}

// New notifier implementation
 class AdvancedNotifier implements NewNotifier {
    @Override
    public void sendInNewFormat(String formattedMessage) {
        // Advanced logic to send the notification
        System.out.println("Sending notification: " + formattedMessage);
    }
}
```

```java
class NotifierBridge extends AdvancedNotifier implements OldNotifier {
    @Override
    public void notify(String message) {
        // Format the message and call the inherited sendInNewFormat
            method
        String formattedMessage = formatMessage(message);
        sendInNewFormat(formattedMessage); // Using method from
            AdvancedNotifier
    }
    private String formatMessage(String message) {
        // Example formatting logic
        return "{\"message\": \"" + message + "\"}";
    }
}
// Client code that uses the old notifier interface
public class NotificationSystem {
    public static void main(String[] args) {
        OldNotifier notifier = new NotifierBridge();
        notifier.notify("System is running");
    }
}
```

The output:

```
Notifying with message: {"notification": "System is operational"}
```